

Capacity-Aware MRP — Capacitated Lot-Sizing with Dixon-Silver Heuristic (v3.26)

Nature of this document: Algorithmic methodology paper describing erpilot's v3.26 **Capacity-Aware MRP** module, which overlays Dixon-Silver (1981) capacity-feasibility heuristic on top of v3.25.10's uncapacitated MRP-II to satisfy work-center capacity constraints. Written in academic paper style.

■ Prerequisite: [MRP_ALGORITHM_DESIGN_EN.md](#) (v3.25.10 uncapacitated MRP-II foundation)

Abstract

This paper describes erpilot v3.26's **Capacity-Aware MRP**, extending v3.25.10's uncapacitated MRP-II to the **Multi-Work-Center Capacitated Lot-Sizing Problem (CLSP)**. Since CLSP is NP-hard [Florian, Lenstra & Rinnooy Kan 1980, *Mgmt Sci* 26], we adopt the **Dixon-Silver (1981) feasibility heuristic**: starting from Wagner-Whitin (1958) uncapacitated optima, when any work-center exceeds capacity in some period, production is shifted earlier to exploit slack capacity in prior periods, accounting for incurred holding-cost penalty. We simultaneously introduce **Karmarkar (1987) setup/run-time decomposition**, **Vollmann et al. (2005) Ch. 7 CRP load profile**, and add new **Routing / RoutingStep data models** as the source of the resource-consumption matrix a_{ik} . Implementation validated via 8 structural-invariant tests and 1 ORM integration test, all 9/9 passing.

Keywords: CLSP, Dixon-Silver heuristic, capacity planning, Routing, bottleneck, operations research

1. Introduction

1.1 The Fundamental Problem with Uncapacitated MRP

v3.25.10's MRP-II (Orlicky-style) answers: "When to order, in what quantity?" But it ignores whether the work-centers can complete it. When Wagner-Whitin lumps orders into a single week, it may yield plans exceeding that week's available hours by 5-10×, severely disconnected from reality. Vollmann et al. (2005) Ch. 7 calls this "Infinite Loading" assumption and notes that every real-world MRP plan must be validated via CRP (Capacity Requirements Planning) before release.

1.2 Why Not Solve as MIP Directly

The CLSP Mixed-Integer Programming form:

$$\begin{aligned} \min \quad & \sum_{i,t} \left[S_i \cdot y_{it} + h_i \cdot I_{it} + c_i \cdot x_{it} \right] \\ \text{s.t.} \quad & I_{i,t} = I_{i,t-1} + x_{it} - d_{it} \quad \& \sum_i \left(a_{ik} \cdot x_{it} + b_{ik} \cdot y_{it} \right) \leq C_{kt} ; \forall k, t \quad \& \text{(capacity)} \quad \& x_{it} \leq M \cdot y_{it} \quad \& \text{(setup link)} \quad \& y_{it} \in \{0,1\}; x_{it} \geq 0; I_{it} \geq 0 \end{aligned}$$

where:

- a_{ik} = run time of item i per unit at work-center k (min/unit)
- b_{ik} = setup time of item i at work-center k (min/batch)
- C_{kt} = available capacity at work-center k in period t (min)

Florian, Lenstra & Rinnooy Kan (1980) proved this problem **NP-hard**, even single-work-center CLSP [Bitran & Yanasse 1982, *Mgmt Sci* 28]. For SMB scale (100-1000 items × 10-20 WCs × 12 weeks), exact MIP runtimes range from seconds to hours, **unsuitable for real-time planning loops**.

1.3 Why Dixon-Silver

Heuristic choices:

Method	Reference	Pros / Cons	Adopted
Dixon-Silver	Dixon & Silver (1981) <i>JOM</i> 2	Natural extension of SM; capacity-feasible; simple $O(V K T^2)$	✓
Trigeiro-Thomas-McClain	TTM (1989) <i>Mgmt Sci</i> 35	Lagrangian relaxation; theoretically tighter; complex	future sprint
Maes-van Wassenhove	MW (1988) <i>EJOR</i> 36	Period-by-period; locally optimal	No (myopic)
Exact MIP (PuLP+CBC)	Pochet-Wolsey (2006)	Optimal; NP-hard slow; solver dep	future baseline

Why Dixon-Silver:

- Natural extension of Silver-Meal (v3.25.10 already implemented) — algorithmic family consistency
- No external MIP solver dependency (preserves open-source purity)
- Runs in <10 ms for SMB scale
- Considered "practitioner's first choice" in literature (Silver, Pyke & Peterson 1998, §13)

2. Methodology

2.1 Routing Data Model

Two new tables in `backend/app/models/production.py`:

```
Routing (master, product-level)
├ id, routing_no, product_id, name, version
├ is_default (Boolean), is_active
└ effective_from, effective_to ← reserved for v3.27 ECO/ECN

RoutingStep (line, ordered)
├ id, routing_id (FK → Routing), sequence_no
├ op_name
├ work_center_id (FK → WorkCenter)
├ setup_time (min/batch) ← Karmarkar 1987 b_ik
├ run_time_per_unit (min/unit) ← Karmarkar 1987 a_ik
├ queue_time, move_time ← informational, not capacity-loading
└ is_critical (bottleneck candidate)
```

Design rationale:

- `Routing` is a **product-level template**, separate from existing `Operation` (which is bound to `production_order`) — same routing reusable across multiple WOs
- `is_default` flag supports multi-version (paves way for v3.27 ECO/ECN)
- `queue_time` / `move_time` excluded from capacity load — per Karmarkar (1987) *Mgmt Sci* 33, these are "lead-time loading" not "capacity loading"

2.2 Resource Profile Computation

For each product i , derive resource consumption from default Routing's RoutingSteps:

$$\text{Profile}(i) = \{ (k, b_{ik}, a_{ik}) \;;; \text{step on WC } k \text{ in default Routing of } i \}$$

Implemented in `build_resource_profile(db)`: reads only `is_default=True AND is_active=True` routings (validated by `test_build_resource_profile_uses_default_routings_only`).

2.3 CRP Load Profile (Capacity Requirements Planning)

Given planned orders x_{it} (from v3.25.10 Wagner-Whitin), compute required minutes per (WC k , period t):

$$L_{kt} = \sum_{i: k \in \text{Profile}(i)} \left[b_{ik} \cdot \mathbb{1}_{x_{it} > 0} + a_{ik} \cdot x_{it} \right]$$

Available capacity:

$$C_{kt} = T_{\text{period}} \cdot \eta_k$$

where T_{period} is total minutes in period (e.g., $8h \times 5d \times 60 = 2400$ min/week) and η_k is work-center efficiency factor [Vollmann 2005 §7.3].

Utilization: $\rho_{kt} = L_{kt} / C_{kt}$. When $\rho_{kt} > 1$, overload.

2.4 Dixon-Silver Feasibility Heuristic

Core idea: shift overloaded period's demand to slack capacity in earlier periods, at the cost of holding-cost penalty.

Algorithm (process overloads backwards from late to early):

```

ALGORITHM: Dixon-Silver Capacity Feasibility
Input:  planned_orders[i][t], Profile, work_centers, horizon T
Output: adjusted_orders, CapacityPlanResult

1. for t = T-1 down to 0:
2.   loads ← compute_CRP(adjusted_orders, Profile)
3.   for each WC k where load[k][t] > capacity[k][t]:
4.     excess ← load[k][t] - capacity[k][t]
5.     contributors ← {(i, x_it, load) : item i loads k in period t}
6.     sort contributors by load desc           # biggest first
7.     for (i, qty, load) in contributors:
8.       if excess ≤ 0: break
9.       qty_to_shift ← min(qty, excess / a_ik)
10.      for offset = 1 .. max_shift:
11.        earlier_t ← t - offset
12.        if earlier_t < 0: break
13.        if load[k][earlier_t] has slack:
14.          adjusted[i][t] -= qty_to_shift
15.          adjusted[i][earlier_t] += qty_to_shift
16.          holding_penalty += qty_to_shift × offset × h
17.          excess -= qty_to_shift × a_ik
18.          break
19.     if excess > 0 (after all contributors):
20.       flag (k, t) as INFEASIBLE

```

Why backwards order: Once period t 's overload is resolved, subsequent capacity checks at $t-1$ reflect "hours borrowed for t ". Dixon & Silver §3.2 prove this order achieves local optimum in a single pass.

Complexity: $O(|V| \cdot |K| \cdot T^2)$ ($|V|$ items, $|K|$ WCs, T periods). For SMB ($|V|=500$, $|K|=10$, $T=12$) $\approx 720,000$ ops ≈ 5 -10 ms.

2.5 Algorithm Invariants

Reviewer-grade invariants:

Invariant	Mathematical statement	Test
Feasibility	$\rho_{kt} \leq 1$ $\forall (k,t) \notin \text{infeasible}$	<code>test_dixon_silver_shifts_to_earlier_period</code>
Demand preservation	$\sum_t \text{adjusted}_{it} = \sum_t \text{planned}_{it}$	<code>test_dixon_silver_demand_preservation_multi_product</code>
Holding cost monotone	Each shift increases holding cost	<code>test_dixon_silver_holding_cost_monotonic</code>
No shift when no slack	If all $t' < t$ are saturated, no shift occurs	<code>test_dixon_silver_infeasible_when_no_earlier_slack</code>

3. Implementation

See `MRP_CAPACITY_AWARE_DESIGN_ZH.md` §3 for code structure and architecture integration diagram.

4. Validation

4.1 Structural Invariant Tests (8 cases)

Test	Validates
compute_load_single_product	Karmarkar formula: $L = b + Q \cdot a$
dixon_silver_no_overload_no_shift	No shifts when not overloaded, penalty=0
dixon_silver_shifts_to_earlier_period	Overload triggers shift, demand preserved
dixon_silver_demand_preservation_multi_product	Multi-product × multi-WC still conserves
dixon_silver_infeasible_when_no_earlier_slack	Overload at $t=0 \rightarrow$ flag
overload_detection_utilization	$\rho > 1 \rightarrow is_overload$
dixon_silver_holding_cost_monotonic	$\Delta \text{cost} = \sum \text{qty} \cdot \Delta t \cdot h$

4.2 ORM Integration Tests (2 cases)

Test	Validates
routing_model_crud	Routing + RoutingStep create/read
build_resource_profile_uses_default_routings_only	Only is_default=True enters profile

4.3 Results

9/9 tests pass at v3.26 release.

5. Limitations and Future Work

Topic	Why not	Future direction
Backward shift (backlogging)	This version only shifts earlier, not allowing delayed delivery	Pochet-Wolsey (2006) backlogging model
Alternate work-centers	<code>alternate_group</code> field exists but unused	Consider alternate WC slack in contributor selection
Setup carryover	Cross-period setup charged twice	Sox-Gao (1999) carryover formulation
Overtime / 3rd shift	No model of overtime cost vs holding cost trade-off	C_{kt}^{OT} as additional variable with premium cost
Stochastic capacity	No machine breakdown / operator absence model	Bertsimas-Thiele (2006) robust optimization
Exact MIP baseline	No exact-solution sanity check	Future sprint: PuLP+CBC comparison
Dynamic rolling horizon	Computes entire horizon at once	Sridharan-Berry (1990) rolling horizon

Edge Cases

- 1. Empty profile:** Products without Routing have zero WC load (demand remains in MRP but capacity unconstrained). Customers should set default Routing for all active products.
- 2. Routing without steps:** Same as above.
- 3. Setup time > period capacity:** Single setup time exceeds C_{kt} → directly infeasible.
- 4. Multi-level sub-assembly capacity:** This version uses "parent product routing" only; sub-assembly's own routing capacity awaits v3.27 nested routing.

6. Legal Notice / 法律聲明

Important: Legal nature of capacity-planning output

The algorithms herein (Dixon-Silver 1981, Karmarkar 1987, Vollmann 2005, etc.) are **operations research public-domain knowledge**. This implementation cites references purely for **academic attribution and reproducibility**, with no proprietary licensing claim.

Scope of responsibility

1. **Planning advisory only:** this module's output (`adjusted_orders` / `load_profile` / `infeasible_periods` / `holding_cost_penalty`) constitutes **algorithm-computed suggestions only, NOT:**
 - Automatic dispatch commands to work-centers
 - Requirements for machine maintenance scheduling
 - Commitments to operator overtime
 - Notifications of delivery date changes to customers
 - Any financial decision
2. **Customer responsibility:** before acting on capacity-feasible plans, qualified production-planning personnel must **manually review:**
 - Each work-center's **actual available hours** vs system-configured `capacity_per_day` × `efficiency` (including current-period maintenance, downtime, shift plans)
 - **Operator skill / certification** constraints (CLSP does not model this)
 - **Material lead-time** alignment with shifted earlier release (CLSP assumes materials available on demand)
 - **QC inspection slots** (QC bottleneck usually outside WC model)
 - **Alternate work-center** capabilities (`alternate_group` field)
3. **Algorithm limitations:**
 - **CLSP is NP-hard:** Dixon-Silver is a **heuristic**, not guaranteed globally optimal; lower-holding-cost feasible schedules may exist that this algorithm doesn't find
 - **Deterministic assumption:** this implementation assumes $C_{\{kt\}}$ is deterministic, no stochastic model for breakdowns / absences
 - **No backlogging:** this version does not permit delivery delay (demand must be met at original period or earlier)
 - **No alternate routing:** product uses only its default Routing; alternate routing for future sprint
4. **Handling infeasible_periods:**
 - When the algorithm reports non-empty `infeasible_periods`, it means "current capacity setting cannot satisfy MPS demand"
 - **Customer decides** the resolution: (a) add capacity (OT / 3rd shift), (b) delay delivery, (c) outsource, (d) reduce MPS demand
 - erpilot is not responsible for handling infeasibility outcomes
5. **Disclaimer:** to the maximum extent permitted by applicable law, erpilot assumes no responsibility for:

- **Machine overload, operator burnout, quality degradation** arising from acting on this algorithm's suggestions
- **Planning inaccuracy** from actual capacity deviation from configured values
- **Erroneous scheduling** from incorrect Routing / WorkCenter input data
- **Planning gaps** from dimensions not modeled (QC, operator skill, outsourcing)
- **Contractual / labor disputes** with third parties (customers, suppliers, operators) acting on this plan

6. **Relationship to v3.25.10 MRP:** this version overlays on v3.25.10's MRP-II result; all limitations and disclaimers in `_MRP_ALGORITHM_DESIGN_EN.md_` §6 also apply cumulatively here.

Recommended practice

- Treat algorithm output as a "**capacity-feasibility check report**" rather than automatic dispatch commands
- Have plant manager / production controller review `infeasible_periods` and decide on overtime / outsourcing / delay
- Maintain alternate routings for critical products (manual switchover possible even without auto-routing support yet)
- Compare actual WC utilization vs planned monthly; feed back into `capacity_per_day` / `efficiency` parameters
- Reserve 10-15% capacity buffer for peak periods (e.g., pre-holiday rushes)

7. References

See `_MRP_CAPACITY_AWARE_DESIGN_ZH.md_` §7 for full reference list (13 entries).

Key references:

- Dixon & Silver (1981) — core heuristic algorithm
- Florian, Lenstra & Rinnooy Kan (1980) — CLSP NP-hardness proof
- Karmarkar (1987) — capacity vs lead-time loading
- Vollmann et al. (2005) — CRP framework
- Trigeiro, Thomas & McClain (1989) — Lagrangian alternative (future)
- Pochet & Wolsey (2006) — MIP exact methods
- Pinedo (2016) — scheduling theory

8. Changelog

Version	Date	Change
v3.25.10	2026-05-20	Uncapacitated MRP-II (Orlicky LLC + Wagner-Whitin)
v3.26	2026-05-20	This release: Capacity-aware MRP — Routing/RoutingStep models + CRP load profile + Dixon-Silver feasibility heuristic
Future v3.27+	TBD	Setup carryover (Sox-Gao 1999) / Alternate routing / ECO-ECN
Future v3.28+	TBD	Lagrangian relaxation (TTM 1989) / Exact MIP baseline (PuLP+CBC)
Future v3.29+	TBD	Stochastic CLSP (machine breakdown, operator absence)

Last updated: 2026-05-20 (v3.26) **Authors:** erpilot engineering team (with IE/OR academic methodology citations) **Version:** 1.0 **Prerequisite:** [MRP_ALGORITHM_DESIGN_EN.md](#) (v3.25.10 uncapacitated MRP-II foundation) **Chinese version:** [MRP_CAPACITY_AWARE_DESIGN_ZH.md](#)